Industry workflow: auditing the Jiaxuan conversation

Two ladders, corpus-dependent transfer, and what Marin actually tested

> **Note**
>
> Detailed companions now cover Step Law, Farseer, Power Lines, Marin optimizer searches and Datakit code. The code audit distinguishes recipe generations and actual call paths, rather than treating all Marin experiments as one setup.

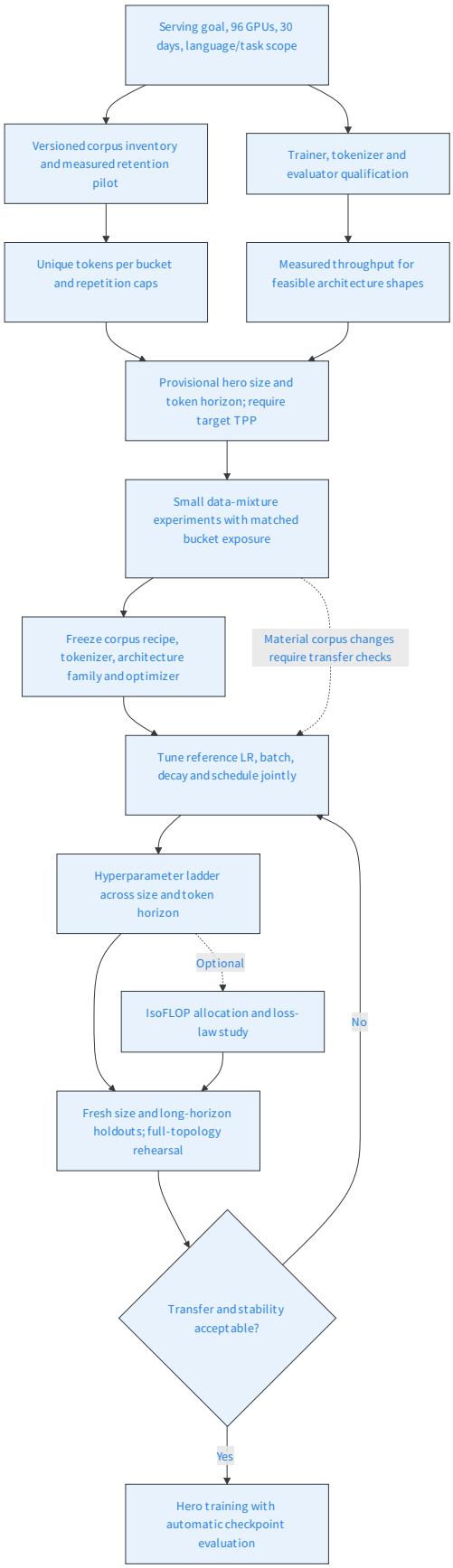
Verdict and reading order

The proposed sequence is a good starting point **after replacing a one-way chain with a constrained, iterative design**. Prioritize a measurable corpus release and a stable trainer/evaluator together. Choose a provisional hero architecture under data, training-time and serving constraints; tune a transferable recipe; validate that recipe at longer horizons and larger sizes. Loss prediction is an additional deliverable, not a prerequisite to every production run.

The conversation notes remain unchanged. They are valuable practitioner testimony, not a published experimental result. This companion audits their claims against Step Law v3, Farseer v3, Power Lines v2, the Marin Delphi report, and actual Datakit mixture records. The papers' PDFs, LaTeX and discovered code are already in `references/`; see the source inventory.

Use this page to resolve the experimental logic, then read data engineering, training and tuning, checkpoint evaluation, and the 96-GPU plan. These pages distinguish observations from proposed engineering choices.

A corrected decision process



This is our synthesis. It does not assert that every lab performs these stages in this exact order. Data experiments need an adequate optimizer; final optimizer transfer needs the final data distribution. Resolve that circularity with a provisional recipe, then retune the finalists. Do not launch a full hyperparameter ladder for each exploratory data edit.

Claim-by-claim audit

Proposed claim	Assessment	Evidence or correction
Determine data size first	Useful priority, incomplete alone	Inventory **unique retained** tokens by bucket, then choose exposures under compute/serving constraints; Datakit records bucket availability and exposures separately
Width must be a power of two	False as a universal requirement	Step Law Table 4 uses widths 960 and 1280; Table 5 uses 1408. Marin's `CompletedAdamHHeuristic` scans width in steps of 128 or 256, enforcing head divisibility
Width determines depth	A recipe choice	Marin derives depth from a width rule; this defines its tested family, not an experimentally established optimum for every width
All small ladder models must only scale width	Too restrictive	A fixed-depth width family isolates width transfer; a family changing depth needs an additional depth-transfer check. MoE expert geometry is another independent axis
Exhaustively tune every small run	Usually too expensive for this allocation	Step Law used 3,700 runs/~1M H800 hours. Marin exhaustively tuned a reference, then used prescriptions and Vizier. We should use bounded searches and expand only when a minimum lies on an edge
Fit LR before batch	Sensible initial coordinate search, insufficient final procedure	Step Law demonstrates coupled LR/batch contours; Power Lines demonstrates batch/LR/weight-decay interactions. Revisit the joint neighborhood after each coordinate pass
LR and batch laws only depend on model size	Unsupported	Step Law fits both size and horizon; Power Lines finds batch dependence primarily on horizon; Delphi's first recipe failed at long horizons
Every mixture change requires a complete new law	Too strong	Step Law explicitly tests transfer across markedly different recipes. Revalidate first; refit if optima or stability shift materially
IsoFLOP experiments are optional	Correct for a fixed constrained hero decision	They become important if the goal is to select compute-optimal allocation or claim a calibrated loss forecast; Delphi actually used IsoFLOP minima
Any corpus works equally well for scaling research	Not established	A fixed corpus permits internal comparisons, but mixture changes can alter rankings, difficulty and transfer. Held-out target-domain evaluation remains necessary
Broad early, high quality late always wins	Hypothesis worth testing	Marin released both fresh and resumed-cooldown mixture experiments. Shared-prefix branches test a conditional late-stage intervention, not an independent full-run replication

Keep four quantities separate

Let U_b be unique retained tokens in bucket b, counted with our frozen tokenizer after corpus-wide deduplication, decontamination and held-out removal. Let D be training token exposures and w_{pb} the token fraction assigned to bucket b in phase p, whose duration is D_p. Then

$$D = \sum_p D_p, \qquad D_b = \sum_p D_p w_{pb}, \qquad e_b = \frac{D_b}{U_b}.$$

An epoch cap $e_b \le e_{b,\max}$ constrains oversampling. Repeated tokens are real training exposures, **not new unique tokens**. An effective-data quantity D_{eff} requires an explicitly fitted repetition model; it must not be silently substituted for U or D.

The notes' use of `U_eff` after counting repetition merges these definitions. Preserve the notes as testimony, but use (U_b, D_b, e_b) in our manifests. Likewise, synthetic rewrites and translations need parent provenance; a different text hash does not establish independent information.

For dense models, $C \simeq 6ND$ is a useful approximation with a declared parameter convention. For MoE and looping, use architecture-specific FLOPs per token $f(\mathcal{A}, \ell)$ at context length ℓ:

$$D \le \frac{G\,t\,r\,q}{f(\mathcal{A}, \ell)}, \qquad \sum_b w_b = 1, \quad D_b \le e_{b,\max} U_b.$$

Here G is GPU count, t is allocated seconds, r is useful model FLOPs per second per GPU, and q is productive availability. Total stored parameters, one-traversal active parameters, and parameters reused across executed visits must be reported separately. Our $D/N_{\text{stored}} \ge 200$ screen expresses the user's overtraining preference; it is not a published MoE optimum.

What Step Law contributes

Published setup. Step Law v3 §3.2 uses a 65,536-token BPE vocabulary, context 2,048, AdamW $(\beta_1, \beta_2) = (0.9, 0.95)$, $\epsilon = 10^{-8}$, decay 0.1, clipping 1, 2,000 warmup steps, and cosine annealing to an **absolute** final LR 10^{-5}. Its LR grid uses powers of two with half-exponent increments; batches range from 32,768 to 4,194,304 tokens. These numerical conventions are part of the result.

Its Table 1 / Eq. 1 report, using raw parameter/token counts and token batches,

$$\eta_\star = 1.79 N^{-0.713} D^{0.307}, \qquad B_\star = 0.58 D^{0.571}.$$

The paper's §1.1 parameter definition is non-embedding/model-body-oriented; its footnote distinguishes an alternative count including the head. Table 5 reports both total and activated MoE counts. **Do not substitute active count, total stored count including embeddings, or a loop execution counter interchangeably.** Reproduce their counting convention before using their tool; fit our MoE family rather than assuming invariance across recurrent depth.

Concrete dense examples from Table 4 include 215M non-vocabulary parameters at width 960/depth 7; 429M at width 1280/depth 10; and 1.074B at width 2048/depth 16. Horizons span different multiples of each size. Table 6 tests a substantial code/math upweighting and an English/Chinese recipe. This is evidence for *testing* recipe transfer, contradicting an unconditional rule that every distribution edit invalidates all HP knowledge.

Interpretation. The positive horizon exponent applies while batch is co-selected under this schedule. It is not inconsistent with a negative horizon correction at fixed batch in another optimizer. Do not combine exponents from different parameterizations.

Artifacts. The official Step Law repository contains `dense_lr_bs_loss.csv`, `moe_lr_bs_loss.csv`, bootstrap fit parameters and `code/fit_tool.py`. A pinned archive is now in `references/optimization/2025-step-law-hyperparameter-scaling/code/step-law--steplaw/`. Its predictor accepts raw N, D and sequence length and converts batch units. These

are fitting artifacts, not proof of a complete reproduced trainer. Our released-table audit counts **1,911 dense rows, 708 MoE rows and 1,000 bootstrap coefficient rows** in the downloaded code snapshot. That is not the same scope as the paper's 3,700 training runs. The CSV `bs` field is in **sequences**: convert using its context length (2,048 in these tables) before comparing to the token-batch formula. Grouping by (N, D) alone is only descriptive if shape, mixture or sparsity differs.

What Power Lines changes about our tuning

Power Lines v2 §2–3 uses AdamW/µP, SlimPajama, a GPT-like ALiBi/SwiGLU architecture, a 1.1B-token fixed validation set, and linear decay with 10% warmup on Cerebras CS-3. It studies overtraining to 1,280 TPP, making the **regime** particularly relevant to our goal, though hardware and model family differ.

The normalized AdamW timescale is

$$\tau = \frac{B}{\eta\lambda D}, \qquad \tau_\star \approx 1.084(D/N)^{-0.527}, \qquad \lambda_\star = \frac{B}{\eta D \tau_\star}.$$

Figure 1 reports $B_{\star,s} = 0.0306 D^{0.383}$ and $B_{\mathrm{crit,s}} = 0.0471 D_{\min}^{0.462}$, with the output batches in **sequences of length 2,048**, while D is tokens. Multiply those outputs by 2,048 to obtain token batches. In the timescale equation above, B must instead be in **tokens**, as Appendix E explicitly states. Distinguish the loss-optimal batch from the critical batch defined by a compute/data-efficiency tradeoff. Appendix units and parameter groups matter before evaluating the coefficients; the exponent is the portable hypothesis, not an unconditional numeric default.

A useful concrete check is §2.4: for 111M at 200 TPP, default loss is 2.810, tuning LR gives 2.808, and tuning decay gives 2.805. Table 1 also compares batch sweeps with LR versus decay retuning. Thus our previous fixed decay of 0.1 is only an initialization for experiments. A LR/batch-only ladder omits a demonstrated degree of freedom.

A µP **base** LR and each parameter group's actual LR are different. The paper's width transfer prescription does not mean setting every group's LR to $1/\mathrm{width}$. Embeddings, output, norm and router groups require their own stated rules.

What Marin actually did: Delphi

The Delphi report describes a recipe, a model suite, and a loss law. The original recipe used Cautious AdamC, width/batch LR transfer, a depth/width relation, approximately fixed step counts and decay fixed at 0.1. Its small IsoFLOP grid looked clean; the 10^{22}-FLOP holdout missed loss by 2.5%, and the 10^{23} run diverged. Multiple changes were subsequently made, so do not attribute the fix to a single isolated causal ablation.

The replacement optimizer was **AdamH**, not MuonH. The final recipe includes a horizon correction and retuned reference settings. At the inspected Marin commit, `experiments/references/completed_adamh.py` specifies

$$\eta_H = 0.00630\sqrt{B_s/64}(2.5 \times 10^9/D)^{0.3},$$

capped at 0.01; B_s is **sequences**, with context 4,096. Scalar Adam groups have a different LR rule. Epsilon and β_2 also scale. The code has a width-dependent depth formula, not fixed depth; it scans widths in multiples of 128 or 256. This file is a full optimizer-specific prescription, not a formula to paste into AdamW.

Delphi fits seven IsoFLOP optima over 3×10^{18}–3×10^{20} FLOPs, with larger held-out budgets up to 10^{23}. The report gives a pre-registered prediction within 0.2% for the 25B/600B-token largest run. This is evidence of a **validated recipe in that setting**, not a 300× transfer guarantee for our new MoE/looped architecture.

What Farseer adds when prediction is required

Farseer v3 §2.2 uses approximately 1,000 models, 65,536 BPE vocabulary, 2,048 context, AdamW, 2,000-step warmup and absolute final LR 10^{-5}. It standardizes HPs using Step Law (Appendix A.1), then studies a two-dimensional loss surface on geometrically spaced sizes and horizons. Its fixed validation set contains 30M tokens; BPC permits a compression-based comparison.

The key distinction from a separable Chinchilla fit is an interaction term:

$$L(N, D) = E + U(N) + V(D) + H(N, D),$$

with the fitted form described in §3 as

$$\widehat{L}(N, D) = f_B(N; \theta_B) D^{-f_A(N;\theta_A)} + f_U(N; \theta_U).$$

Its Eq. 3 makes the functional dependence explicit:

$$\widehat{L}(N, D) = e^{a_3 N^{\gamma} + b_3} + e^{a_2 N^{\beta} + b_2} D^{-e^{a_1 N^{\alpha} + b_1}}.$$

The nine fitted coefficients/exponents are properties of a particular training/evaluation suite; fitting them to a single ray or a handful of noisy endpoints would be underconstrained. Compare held-out errors with a simpler separable baseline before accepting the added flexibility.

Thus the data exponent may depend on model size. Its released implementation is archived locally. Fitting a flexible surface needs genuinely two-dimensional data and held-out sizes/horizons; a single fixed-TPP ray does not identify a general allocation law. Treat the authors' improvements as results on their suite, not certified error bars for ours.

Our executable experimental decisions

1. Freeze a release of source revisions, tokenizer, normalization, dedup and eval exclusions. Estimate retained U_b from stratified pilots; uncertainty belongs in the inventory.
2. Benchmark plausible shapes before making the hero parameter count final. Include the real packer, router and input pipeline.
3. Tune one reference recipe with bounded joint LR/batch/decay search. Compare optimizers only at equally tuned, fully annealed endpoints.
4. Screen data recipes, then recheck optimizer transfer on finalists. Freeze the final mixture before expensive transfer fits.
5. Use at least three widths and several token horizons with variation in both axes. Add a pure width-transfer family and a depth-changing holdout. For the overtrained target, put at least one fresh holdout around 200–250 TPP; a 10–80 TPP-only ladder leaves the target outside the validated regime.
6. Fit log-linear HP hypotheses, retain measured near-optimal plateaus, and propagate uncertainty. Minima on search boundaries trigger expansion; unstable runs are outcomes, not omitted points.
7. Validate predicted HPs at an unseen size/horizon with a local $0.5, 1, 2$ LR bracket and neighboring feasible batches. A broad stable plateau is preferable to an extrapolated sharp optimum.
8. Run an IsoFLOP/loss study only if its allocation/prediction value justifies its budget. If skipped, report that no calibrated hero-loss forecast exists.

The quantitative plan and acceptance criteria are in the revised budget guide. None of these experiments has been run here.